Mini Project 1: Data Mining & Machine Learning

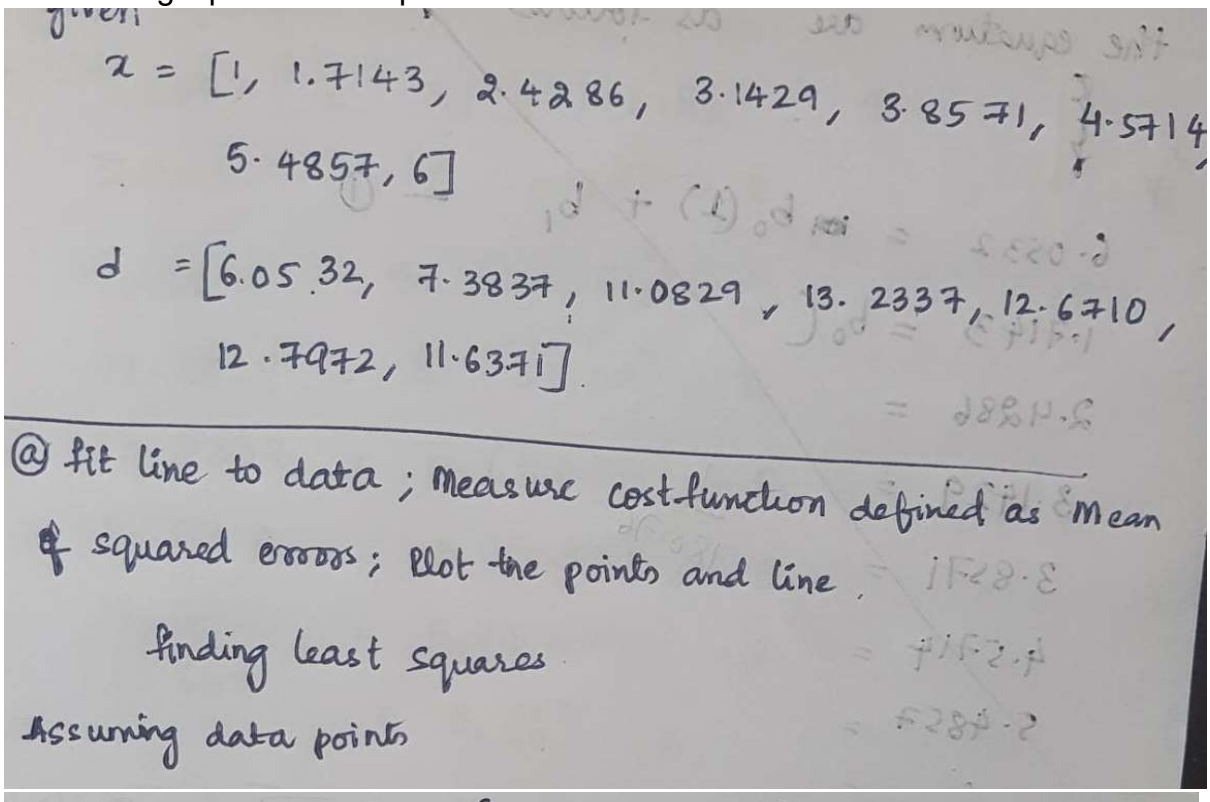
Name: Imtiyaz Mohammed

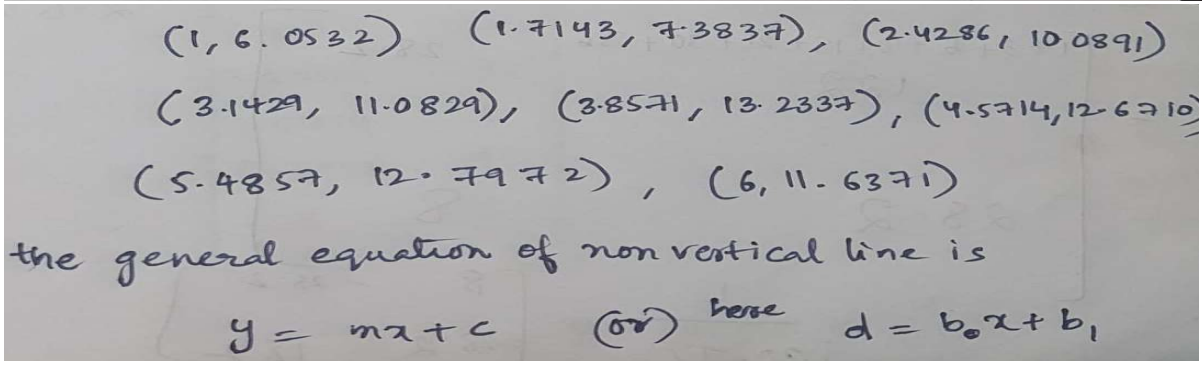
Z23749265

Question 1.

a. **A system has generated outputs marked by vector 'd' in response to inputs which are listed in vector'x':**
d = [6.0532, 7.3837, 10.0891, 11.0829, 13.2337, 12.6710, 12.7972, 11.6371] ;
x = [1 , 1.7143, 2.4286 , 3.1429, 3.8571, 4.5714, 5.4857, 6] ; Use the theory of regression to fit a line to this data. Measure the cost function defined as the mean of squared errors. Plot your data points and the line that models the system's function.

Considering input X and output d





The equations are as follows

$$6.0532 = B_0(1) + B_1$$
$$7.3837 = B_0(1.7143) + B_1$$
$$10.0891 = B_0(2.4286) + B_1$$
$$11.0829 = B_0(3.1429) + B_1$$
$$13.2337 = B_0(3.8571) + B_1$$
$$12.6710 = B_0(4.5714) + B_1$$
$$12.7972 = B_0(5.4857) + B_1$$
$$11.6371 = B_0(6) + B_1$$

$$\therefore A = \begin{bmatrix} 1 & 1 \\ 1.7143 & 1 \\ 2.4286 & 1 \\ 3.1429 & 1 \\ 3.8571 & 1 \\ 4.5714 & 1 \\ 5.4857 & 1 \\ 6 & 1 \end{bmatrix}_{8\times 2} \quad X = \begin{bmatrix} B_0 \\ B_1 \end{bmatrix}_{2\times 1} \quad B = \begin{bmatrix} 6.0532 \\ 7.3837 \\ 10.0891 \\ 11.0829 \\ 13.2337 \\ 12.6710 \\ 12.7972 \\ 11.6371 \end{bmatrix}_{8\times 1}$$

$$AX = B \Rightarrow X = A^{+}B$$

$$X = A^{+}B = (A^TA)^{-1}A^T \cdot B$$

$$\Rightarrow \begin{bmatrix} B_0 \\ B_1 \end{bmatrix} = \begin{bmatrix} 12 & 28.2 \\ 28.2 & 8 \end{bmatrix}^{-1} \cdot A^T \cdot B$$

$$\begin{bmatrix} B_0 \\ B_1 \end{bmatrix} = \begin{bmatrix} \frac{1}{180.76} \begin{bmatrix} 8 & -28.2 \\ -28.2 & 122 \end{bmatrix} \end{bmatrix} \begin{bmatrix} 1 & 1.71 & 2.42 & 3.14 & 3.8 & 4.5 & 5.4 & 6 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \times B$$

$$= \frac{1}{180.76} \begin{bmatrix} -20.2 & -14.4856 & -8.7712 & -3.0568 & 2.6568 & 8.3712 & 15.6856 \\ 93.8 & 73.6569 & 53.51348 & 33.3702 & 13.2297 & -6.9134 & -32.6967 \end{bmatrix}$$

$$\begin{matrix} 19.8 \\ -47.2 \end{matrix}$$

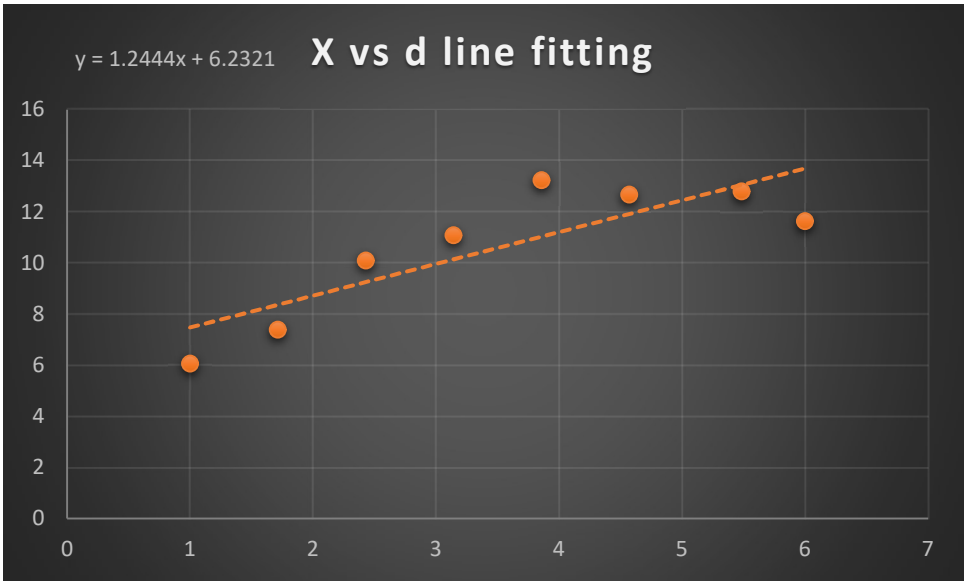
$$\times B \begin{bmatrix} 6.0532 \\ 7.3837 \\ 10.0891 \\ 11.0829 \\ 13.2337 \\ 12.6710 \\ 12.7972 \\ 11.6371 \end{bmatrix}$$

$$= \frac{1}{180.76} \begin{bmatrix} 220.7734 \\ 1150.77 \end{bmatrix} = \begin{bmatrix} 1.2243 \\ 6.2366 \end{bmatrix}$$

$$\begin{bmatrix} B_0' \\ B_1 \end{bmatrix} = \begin{bmatrix} 1.22 \\ 6.236 \end{bmatrix}$$

$\therefore y = 1.22x + 6.236$

$y_{intercept} = 6.236$ at $x = 0$



Based on the equation the predicted d values are as follows

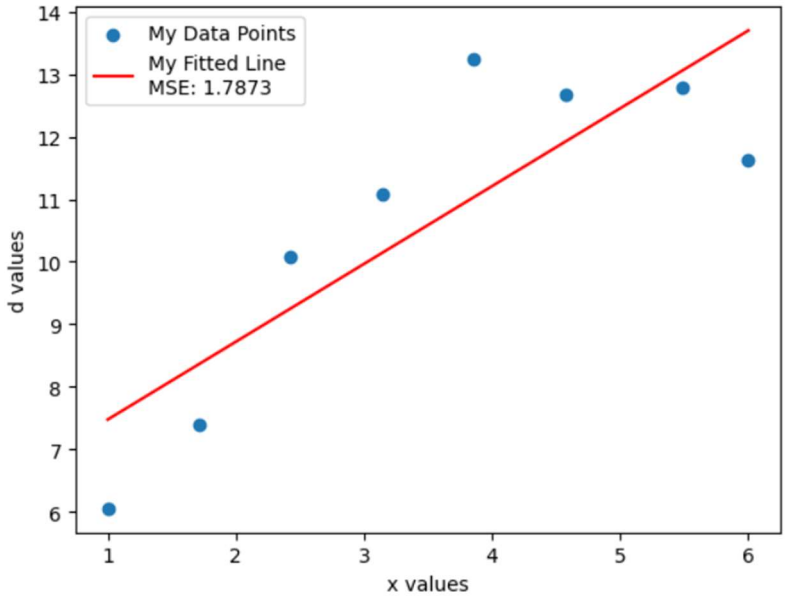
X	actual d	predicted outputs d	(d predicted - d actual)	squared value
1	6.0532	7.4763	1.4231	2.02521361
1.7143	7.3837	8.36510349	0.98140349	0.96315281
2.4286	10.0891	9.25390698	-0.83519302	0.697547381
3.1429	11.0829	10.14271047	-0.94018953	0.883956352
3.8571	13.2337	11.03138953	-2.20231047	4.850171406
4.5714	12.671	11.92019302	-0.75080698	0.563711121
5.4857	12.7972	13.05785651	0.26065651	0.067941816
6	11.6371	13.6978	2.0607	4.24648449
				14.29817899=sum

The mean squared error will be 14.2981/8 =1.7872

Python code for the same:

```
import numpy as np
import matplotlib.pyplot as plt
x = np.array([1, 1.7143, 2.4286, 3.1429, 3.8571, 4.5714, 5.4857, 6])
d = np.array([6.0532, 7.3837, 10.0891, 11.0829, 13.2337, 12.6710, 12.7972,
11.6371])
def slope_intercept(x, d):
    n = len(x)
    mean_x = np.mean(x)
    mean_d = np.mean(d)
    numerator = np.sum((x - mean_x) * (d - mean_d))
    denominator = np.sum((x - mean_x)**2)
    a = numerator / denominator
    b = mean_d - a * mean_x
    return a, b
a, b = slope_intercept(x, d)
```

```
# Generating a range of x values using linspace
x_values = np.linspace(1, 6, 5)
# Calculating predicted y values for the linspace x values
d_predicted_values = a * x_values + b
# Calculating Mean Squared Error (MSE)
mse = np.mean((d - (a * x + b))**2)
# Plotting the data points and the fitted line
plt.scatter(x, d, label='My Data Points')
plt.plot(x_values, d_predicted_values, color='red', label=f'My Fitted Line\nMSE:
{mse:.4f}')
plt.xlabel('x values')
plt.ylabel('d values')
plt.legend()
plt.show()
print(f'My Mean Squared Error (MSE): {mse:.4f}')
```




```
My Mean Squared Error (MSE): 1.7873
```

b. Use the same data but this time fit a second order polynomial to these data points. What's the value of your cost function? Plot the second order curve.

Y=AX^2 +BX+C

6.0532 = a+b+c,

7.3837= 3a+1.7143b +c,

10.0891= 6a+2.4286b +c,

11.0829=10 a+3.1429b+c,

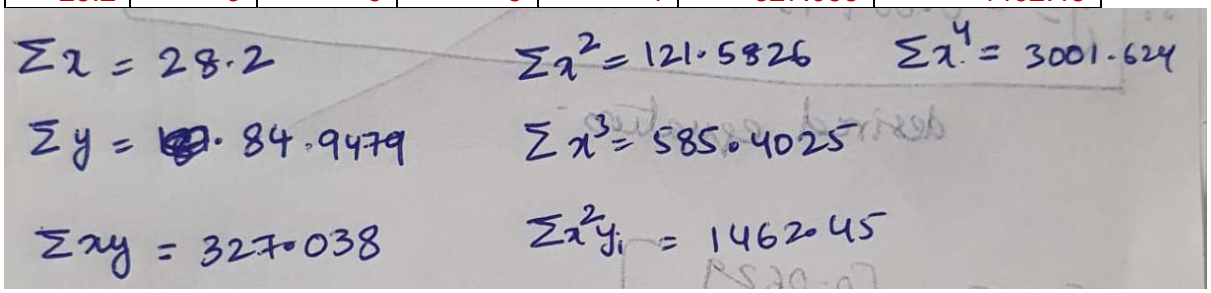
13.2337 =15a+3.8571b+c,

12.6710 =21a+4.5714b+c,

12.7972 =30a+5.4857b+c,

11.6371 =36a+6b+c,

X	D or y	x^2	X^3	x4	xy	x2y
1	6.0532	1	1	1	6.0532	6.0532
1.7143	7.3837	2.938824	5.038027	8.636689	12.65788	21.6994
2.4286	10.0891	5.898098	14.32412	34.78756	24.50239	59.5065
3.1429	11.0829	9.87782	31.045	97.57134	34.83245	109.4749
3.8571	13.2337	14.87722	57.38293	221.3317	51.0437	196.8807
4.5714	12.671	20.8977	95.53174	436.7138	57.92421	264.7947
5.4857	12.7972	30.0929	165.0806	905.5829	70.2016	385.1049
6	11.6371	36	216	1296	69.8226	418.9356
28.2	84.9479	121.5826	585.4025	3001.624	327.038	1462.45



84.9479=8a+28.2b+121.5826c

327.038=585.4025c+121.5826b+28.2a

1462.45=3001.624c+585.4025b+121.5826a

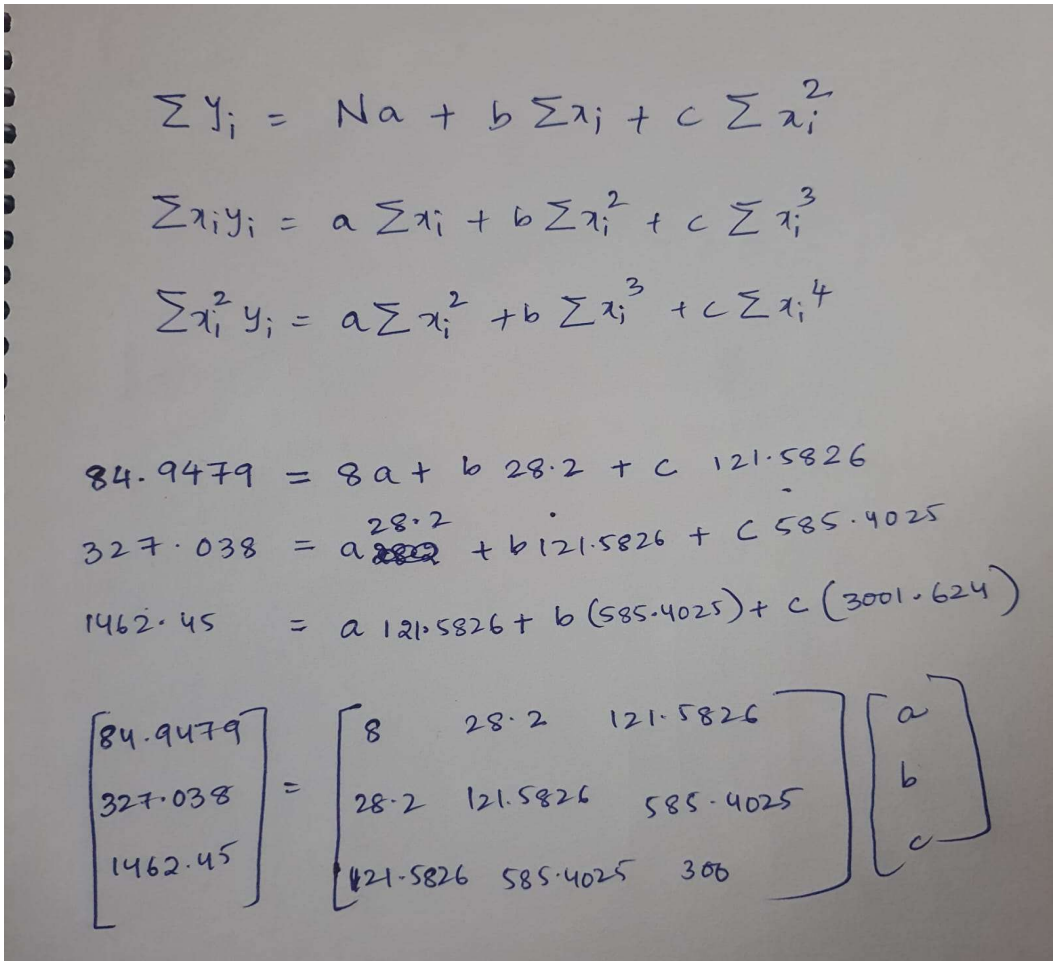
We can represent the system in matrix form AX=B

Ax=B, where:

$$A = \begin{bmatrix} 8 & 28.2 & 121.5826 \\ 28.2 & 121.5826 & 585.4025 \\ 121.5826 & 585.4025 & 3001.624 \end{bmatrix}$$

$$B = \begin{bmatrix} 84.9479 \\ 327.038 \\ 1462.45 \end{bmatrix}$$

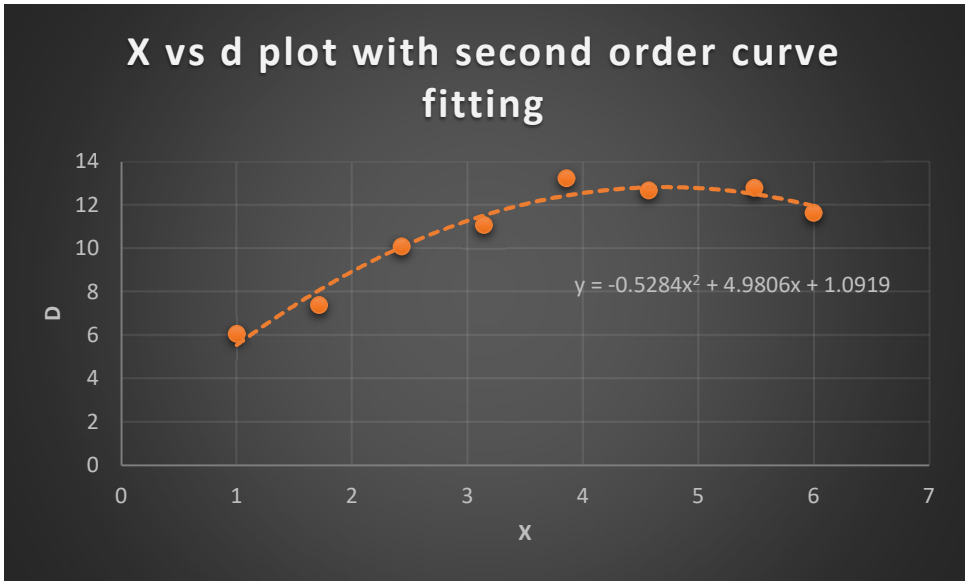
$$x = \begin{bmatrix} a \\ b \\ c \end{bmatrix}$$



Therefore the equation for this becomes new Y' = 1.0919 + 4.9806X-0.5284x2

X	d or y	y'(predicted)	y'-y	(y-y')²
1	6.0532	5.54408	-0.50912	0.259203
1.7143	7.3837	8.07720894	0.693509	0.480955
2.4286	10.0891	10.0711122	-0.01799	0.000324
3.1429	11.0829	11.5257899	0.44289	0.196151
3.8571	13.2337	12.4411515	-0.79255	0.628133
4.5714	12.671	12.8174533	0.146453	0.021449
5.4857	12.7972	12.5122848	-0.28492	0.081177
6	11.6371	11.95238	0.31528	0.099401
sum of mean square =>				1.766793

MSE= 1.766793/8 = 0.22



Python code for the same:

```
import numpy as np

import matplotlib.pyplot as plt

x = np.array([1, 1.7143, 2.4286, 3.1429, 3.8571, 4.5714, 5.4857, 6])

d = np.array([6.0532, 7.3837, 10.0891, 11.0829, 13.2337, 12.6710, 12.7972, 11.6371])

coefficients = np.polyfit(x, d, 2)

# Createing a polynomial function using the coefficients

poly_function = np.poly1d(coefficients)

# Generating a range of x values using linspace

x_values = np.linspace(min(x), max(x), 100)

# Calculating predicted y values  & MSE for the second-order polynomial

d_predicted_values = poly_function(x_values)

mse = np.mean((d - poly_function(x))**2)

print(f'My Mean Squared Error (MSE) for Second Order Polynomial: {mse:.4f}')

# Plotting

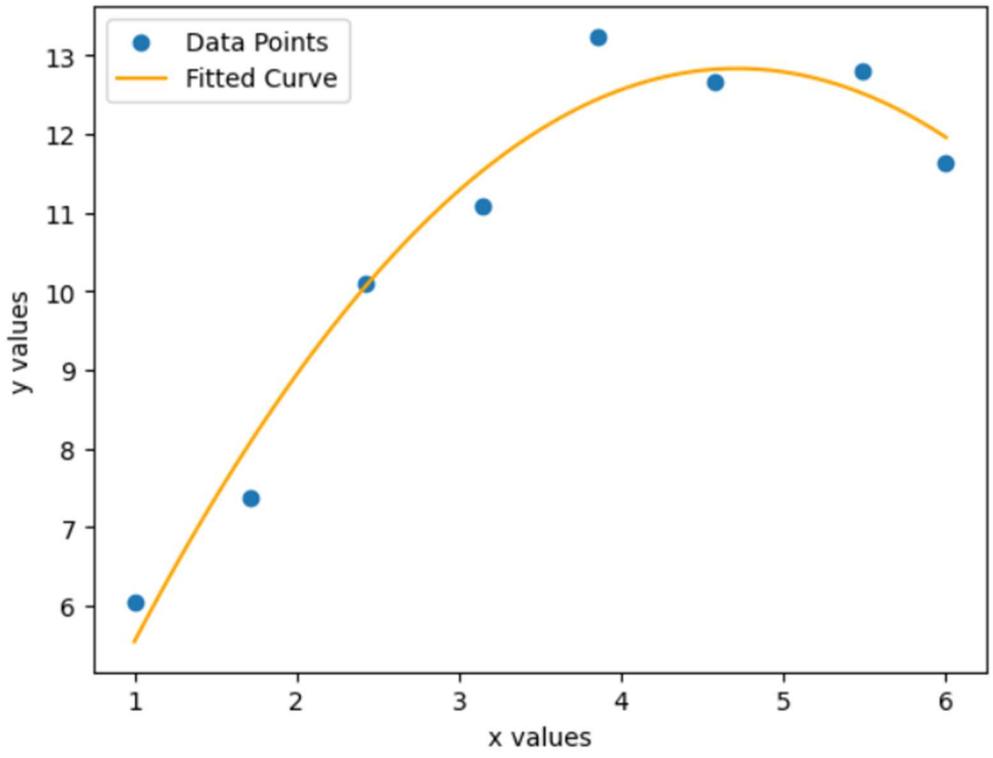
plt.scatter(x, d, label='My Data Points')

plt.plot(x_values, d_predicted_values, color='orange', label=f'Second Order Polynomial\nMSE: {mse:.4f}')

plt.xlabel('x values')

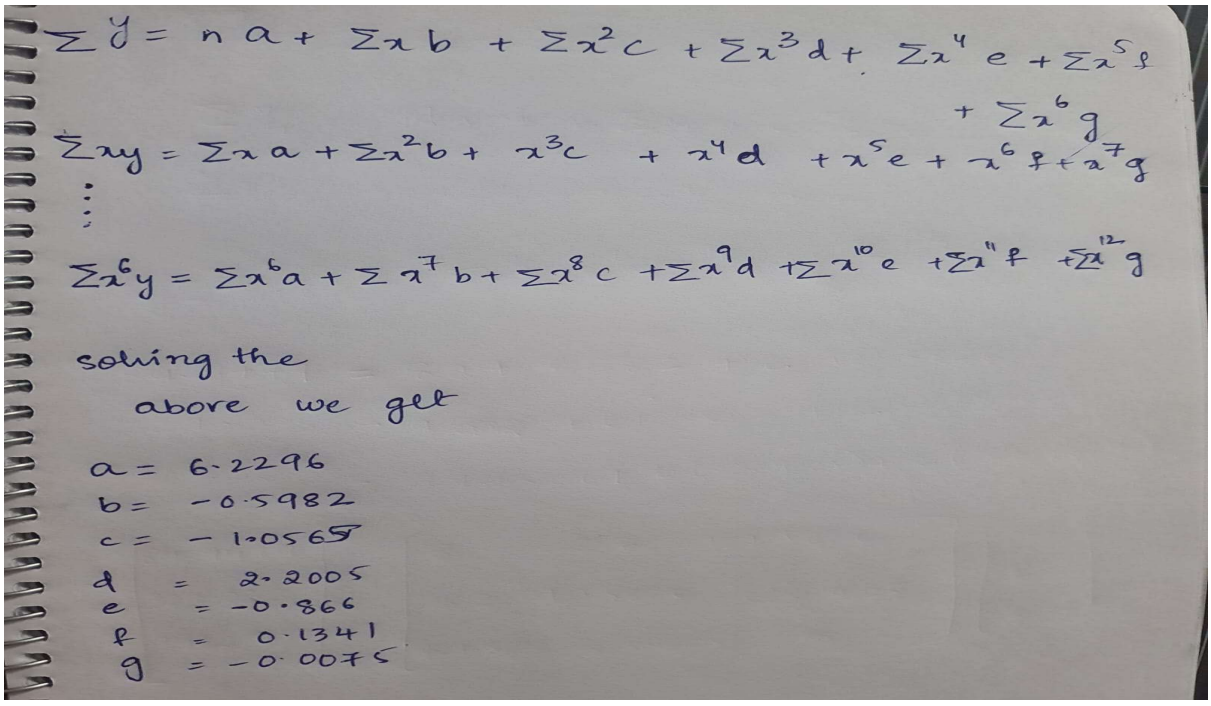
plt.ylabel('d values')
```

```
plt.legend()
plt.show()
```




```
Mean Squared Error (MSE) for Fitted Curve: 0.2208
```

c. Increase the order of the polynomial to 6 and fit the curve. How much error do you measure this time? Plot the 6th order curve.
Now for the 6th order again



$$y = -0.0075x^6 + 0.1341x^5 - 0.866x^4 + 2.2005x^3 - 1.0565x^2 - 0.5982x + 6.2296$$

X	d actual	y'	y'-y	$(y-y')^2$
1	6.0532	6.036	-0.0172	0.000296
1.7143	7.3837	7.50114788	0.117448	0.013794
2.4286	10.0891	9.73026628	-0.35883	0.128762
3.1429	11.0829	11.6256056	0.542706	0.294529
3.8571	13.2337	12.5873489	-0.64635	0.41777
4.5714	12.671	12.7090296	0.03803	0.001446
5.4857	12.7972	11.969018	-0.82818	0.685885
6	11.6371	10.42	-1.2171	1.481332

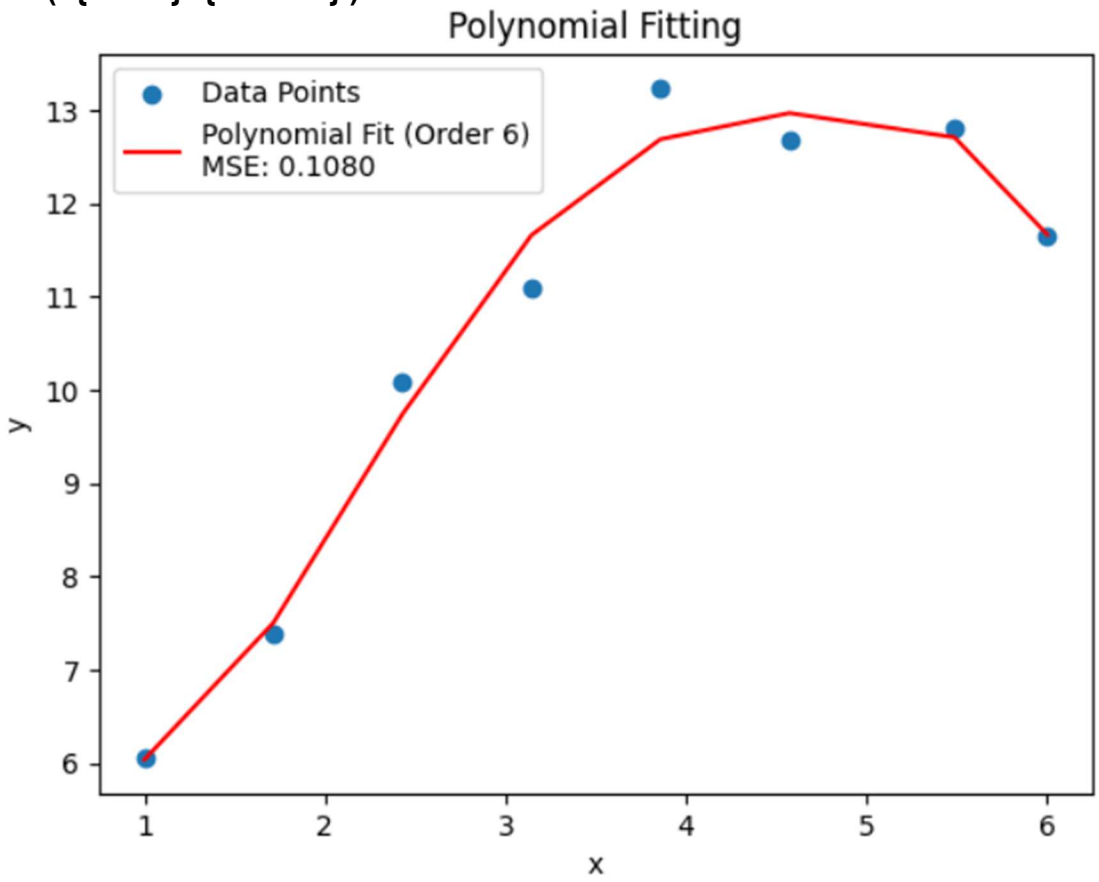
Mse = 0.1080



Python code for the same:

```
import numpy as np
import matplotlib.pyplot as plt
x = np.array([1, 1.7143, 2.4286, 3.1429, 3.8571, 4.5714,5.4857, 6])
d = np.array([6.0532, 7.3837, 10.0891, 11.0829, 13.2337, 12.6710,12.7972,
11.6371])
coefficients = np.polyfit(x, d, 6)
d_new = np.polyval(coefficients, x)
mse = np.mean((d - d_new)**2)
plt.scatter(x, d, label='Data Points')
plt.plot(x, d_new, label=f'Polynomial Fit (Order 6)\nMSE: {mse:.4f}',
color='red')
plt.title('Polynomial Fitting')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
```

```
plt.show()
print(f'Mean Squared Error (MSE): {mse:.4f}')
coeff_names = ['a', 'b', 'c', 'd', 'e', 'f', 'g']
for name, coef in zip(coeff_names, coefficients):
    print(f'{name}: {coef:.4f}')
```




```
Mean Squared Error (MSE): 0.1080
a: -0.0075
b: 0.1341
c: -0.8660
d: 2.2005
e: -1.0565
f: -0.5982
g: 6.2296
```

d. Remove one data point (d: 12.7772, x: 5.2857) from your set. Once again fit the 6th order polynomial. After finding the polynomial bring that data point back and once again measure the cost function for all data points. How much the value of the cost function changed compared to the 6th order polynomial where all data points were used for curve fitting? Is the 6th order polynomial a case of over fitting? Plot the new 6th order curve.

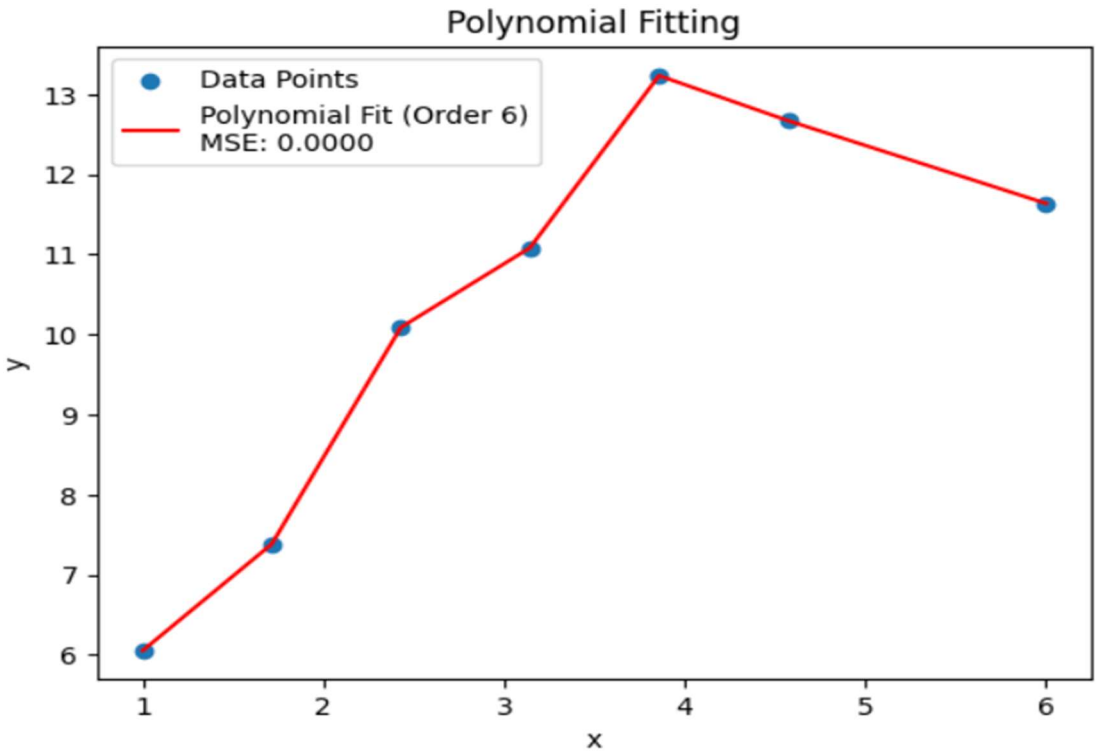
The equation becomes : $y = 0.1996x^6 - 3.9049x^5 + 30.204x^4 - 117.78x^3 + 242.5x^2 - 244.78x + 99.616$

X	d actual	y'	y'-y	$(y-y')^2$
1	6.0532	6.0547	0.0015	2.25E-06

1.7143	7.3837	7.38907431	0.005374	2.89E-05
2.4286	10.0891	10.1088352	0.019735	0.000389
3.1429	11.0829	11.1408914	0.057991	0.003363
3.8571	13.2337	13.3780253	0.144325	0.02083
4.5714	12.671	12.9910784	0.320078	0.10245
6	11.6371	12.8752	1.2381	1.532892

Python code for the same :

```
import numpy as np
import matplotlib.pyplot as plt
x = np.array([1, 1.7143, 2.4286, 3.1429, 3.8571, 4.5714, 6])
d = np.array([6.0532, 7.3837, 10.0891, 11.0829, 13.2337, 12.6710, 11.6371])
coefficients = np.polyfit(x, d, 6)
d_new = np.polyval(coefficients, x)
mse = np.mean((d - d_new)**2)
plt.scatter(x, d, label='Data Points')
plt.plot(x, d_new, label=f'Polynomial Fit (Order 6)\nMSE: {mse:.4f}', color='red')
plt.title('Polynomial Fitting')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.show()
print(f'Mean Squared Error (MSE): {mse:.4f}')
coeff_names = ['a', 'b', 'c', 'd', 'e', 'f', 'g']
for name, coef in zip(coeff_names, coefficients):
   print(f'{name}: {coef:.4f}')
```




```
Mean Squared Error (MSE): 0.0000
a: 0.1996
b: -3.9049
c: 30.2042
d: -117.7825
e: 242.5046
f: -244.7840
g: 99.6163
```

The difference between 6th order with and without one data point = 0.1089-0.000 = 0.1

e. **Plot the value of the cost function as a function of the polynomial order (from 1 to 10) using all data points. Based on this curve, which order is suitable for this dataset to avoid over or under fitting?**

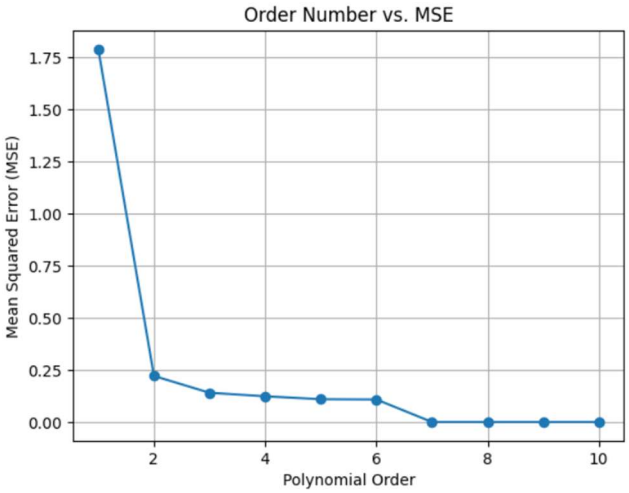
Python code:

```
import numpy as np
import matplotlib.pyplot as plt
d = np.array([6.0532, 7.3837, 10.0891, 11.0829, 13.2337, 12.6710, 12.7972, 11.6371])
x = np.array([1, 1.7143, 2.4286, 3.1429, 3.8571, 4.5714, 5.4857, 6])

# to store all values of mse i have taken an array
mse_values = []
for order in range(1, 11):
    coefficients = np.polyfit(x, d, order)
    predicted_values = np.polyval(coefficients, x)
    mse = np.mean((d - predicted_values)**2)
    mse_values.append(mse)
```

```
print("MSE Array:", mse_values)
plt.plot(range(1, 11), mse_values, marker='o')
plt.title('Order Number vs. MSE')
plt.xlabel('Polynomial Order')
plt.ylabel('Mean Squared Error (MSE)')
plt.grid(True)
plt.show()
```

```
MSE Array: [1.787272257224746, 0.220848070531249, 0.14012432434799876, 0.12330318419215325, 0.10927899894287552, 0.10800458701011581, 2.370089258100044
2e-21, 5.0631740205814085e-22, 7.544342787182966e-23, 5.025901162238981e-24]
```



Over fitting means it is a modelling error that occurs when the function is too closely aligned to a limited set of data pointsTo identify a suitable order that avoids overfitting, we have look for the point where the MSE stops decreasing significantly. In this case, it might be around the 6th order, where the MSE becomes very close to zero. MSE values, the 6th order polynomial seems to be a suitable choice, as it achieves a low MSE.

A model that performs well not only on the training data but also on unseen data is desirable, choosing the 3rd order polynomial regression based on a trade-off between fitting the data well, avoiding overfitting, and achieving a reasonably low cost function (MSE) is reasonable

The MSE value for the 3rd order polynomial is 0.140, which is relatively low and suggests a good fit to the data.

---

2) **Consider a 2-dimensional classification dataset with the given desired values for each point. x1 = [0.5, 0.8, 0.9, 1.0, 1.1, 2.0, 2.2, 2.5, 2.8, 3.0];**
x2 = [0.5, 0.2, 0.9, 0.8, 0.3, 2.5, 3.5, 1.8, 2.1, 3.2];
d = [0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 1.0, 1.0, 1.0, 1.0];
Here for (x1=0.5, x2=0.5) the desired value is 0.0 and for (x1=3.0, x2=3.2) the desired value is 1.0.

a. Use the logistic regression algorithm to design a supervised classifier that can perfectly separate these two sets. Plot these data points and your decision line.

```
import matplotlib.pyplot as plt
import numpy as np
x1 = [0.5, 0.8, 0.9, 1.0, 1.1, 2.0, 2.2, 2.5, 2.8, 3.0]
x2 = [0.5, 0.2, 0.9, 0.8, 0.3, 2.5, 3.5, 1.8, 2.1, 3.2]
labels = [0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 1.0, 1.0, 1.0, 1.0]
data_points = np.array(list(zip(x1, x2)))
class_labels = np.array(labels)
class LogisticRegression:

    def __init__(self):
        self.learning_rate = None
        self.iterations = None
        self.weights = None
        self.bias = None

    def train(self, learning_rate, iterations, features, labels):
        self.learning_rate = learning_rate
        self.iterations = iterations
        num_samples, num_features = features.shape
        self.weights = np.zeros(num_features)
        self.bias = 0

        for _ in range(self.iterations):
            linear_pred = np.dot(features, self.weights) + self.bias
            predictions = self.sigmoid(linear_pred)

            gradient_weights = np.dot(features.T, (predictions - labels)) / num_samples
            gradient_bias = np.sum(predictions - labels) / num_samples

            self.weights -= self.learning_rate * gradient_weights
            self.bias -= self.learning_rate * gradient_bias

    def predict(self, features):
        linear_pred = np.dot(features, self.weights) + self.bias
        predictions = self.sigmoid(linear_pred)
        class_predictions = (predictions > 0.5).astype(int)
        return class_predictions

    def sigmoid(self, x):
        return 1 / (1 + np.exp(-x))

def calculate_accuracy(true_labels, predicted_labels):
    return np.sum(true_labels == predicted_labels) / len(predicted_labels)
def plot_decision_boundary(model, x_values):
    slope = -(model.weights[0] / model.weights[1])
```

```
    intercept = -(model.bias / model.weights[1])

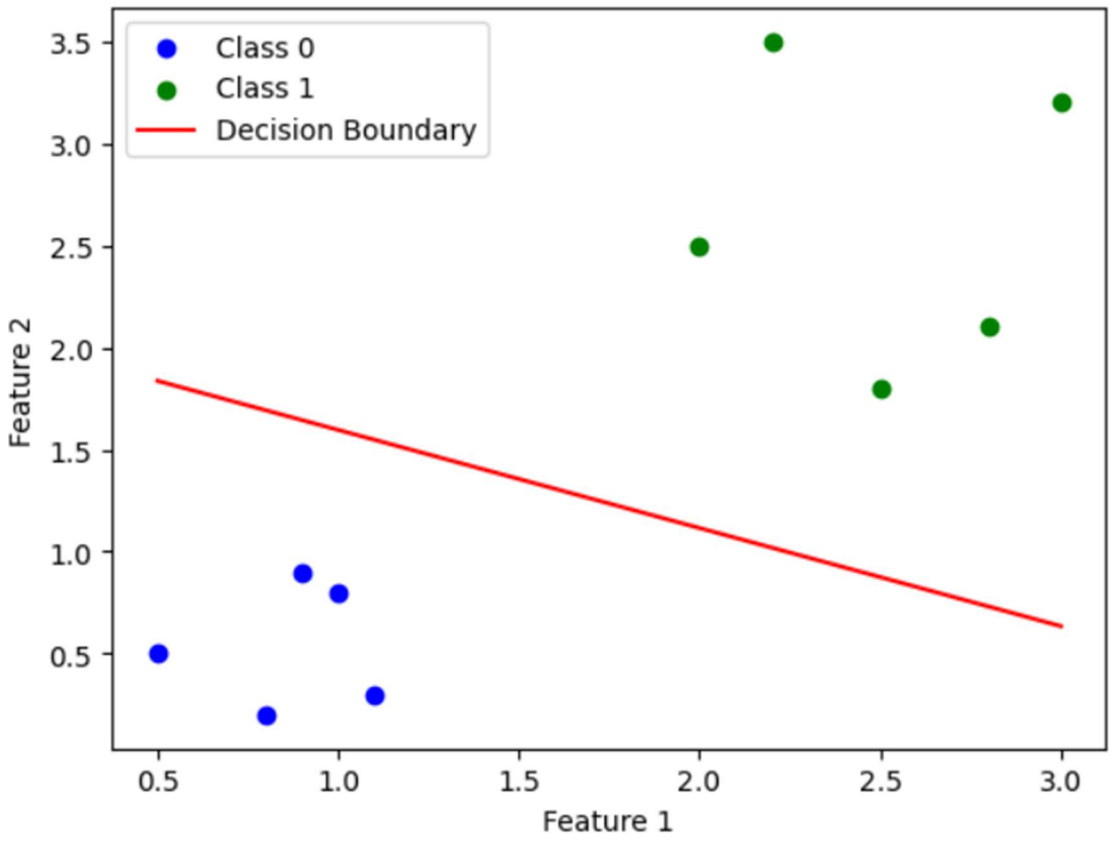
    decision_line_x = np.array([0.5, 3.0])
    decision_line_y = slope * decision_line_x + intercept

    plt.scatter(x_values[:5, 0], x_values[:5, 1], color="blue", label='Class 0')
    plt.scatter(x_values[5:, 0], x_values[5:, 1], color="green", label='Class 1')

    plt.plot(decision_line_x, decision_line_y, color='red', label='Decision Boundary')
    plt.xlabel('Feature 1')
    plt.ylabel('Feature 2')
    plt.legend()
    plt.show()

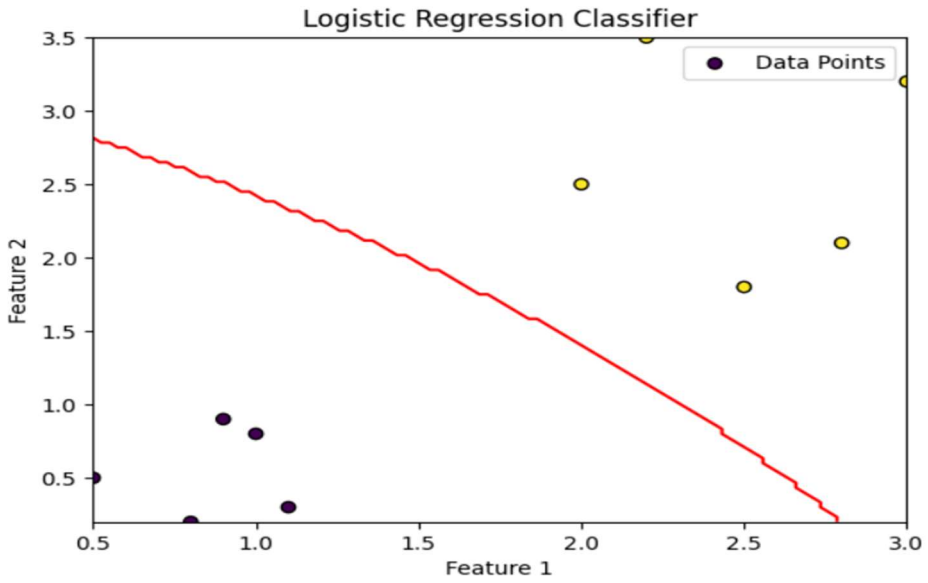
# Creating and training the logistic regression model
logistic_model = LogisticRegression()
logistic_model.train(0.1, 1000, data_points, class_labels)
# Making predictions and calculating accuracy
predictions = logistic_model.predict(data_points)
accuracy = calculate_accuracy(class_labels, predictions)
print(f'Accuracy: {accuracy:.4f}')
plot_decision_boundary(logistic_model, data_points)
```

```
Accuracy: 1.0000
```



b. Increase the order of the decision making curve to 2 and once again use the logistic regression to design the classifier. How do you compare these two classifiers

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LogisticRegression
x1 = np.array([0.5, 0.8, 0.9, 1.0, 1.1, 2.0, 2.2, 2.5, 2.8, 3.0])
x2 = np.array([0.5, 0.2, 0.9, 0.8, 0.3, 2.5, 3.5, 1.8, 2.1, 3.2])
d = np.array([0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 1.0, 1.0, 1.0, 1.0])
X = np.column_stack((x1, x2))
# Increasing the order of the decision-making curve to 2
degree = 2
poly = PolynomialFeatures(degree)
X_poly = poly.fit_transform(X)
# Creating and fitting the logistic regression model
model = LogisticRegression()
model.fit(X_poly, d)
# Plotting the decision boundary
x1_range = np.linspace(min(x1), max(x1), 100)
x2_range = np.linspace(min(x2), max(x2), 100)
X1, X2 = np.meshgrid(x1_range, x2_range)
X_range = np.column_stack((X1.flatten(), X2.flatten()))
X_range_poly = poly.transform(X_range)
predictions = model.predict(X_range_poly)
predictions = predictions.reshape(X1.shape)
# original data point ploting
plt.scatter(x1, x2, c=d, cmap='viridis', marker='o', label='Data Points')
# Plot the decision boundary
plt.contour(X1, X2, predictions, levels=[0.5], colors='red', label='Decision
Boundary (degree=2)')
plt.title('Logistic Regression Classifier')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.show()
```



In the case of linear logistic regression, the equation used to make predictions is quite straightforward i.e.,:

$$\frac{1}{1+e^{-(\beta_0+\beta_1 X_1+\beta_2 X_2)}}$$

Unlike its non-linear regression for this dataset, there is no need to transform or normalize input values; they can be used as they are. The equation involves only two features, X1,X2, making it simpler to work with. The resulting decision boundary is a straight line, which means the separation between different classes resembles a line when visualized on a graph.

On the other hand, non-linear logistic regression introduces a more complex equation:

$$\frac{1}{1+e^{-(\beta_0+\beta_1 X_1+\beta_2 X_2+\beta_3 X_1^2+\beta_4 X_2^2+\beta_5 X_1 X_2)}}$$

The decision boundary is represented as a contour, resulting in a curved separation between classes when visualized on a graph.

Because of the above said complications, I prefer linear regression for this dataset.